Coping with limitations



Running Times: polynomial vs super-polynomial

	10	20	30	40	50	60
n	.00001 second	.00002 second	.00003 second	.00004 second	.00005 second	.00006 second
n^2	.0001 second	.0004 second	.0009 second	.0016 second	.0025 second	.0036 second
n^3	.001 second	.008 second	.027 second	.064 second	.125 second	.216 second
n^5	.1 second	3.2 second	24.3 second	1.7 minutes	5.2 minutes	13.0 minutes
2^n	.001 second	1.0 second	17.9 minutes	12.7 days	35.7 years	36 600 years
3^n	.059 second	58 minutes	6.5 years	385 500 years	2×10^{10} years	1.3×10^{15} years

Largest Problem Solvable in Unit Time

time complexity	current computer	1000 times faster	1 000 000 times faster	1 000 000 000 times faster
n	100	100 000	100 000 000	100 000 000 000
n^2	10	316	10 000	316 227
n^3	4.6	46	464	4 641
n^5	2.5	10	39.8	158
2^n	6.6	16	26	36.5
3^n	4.2	10.5	16.8	23.0

How to tackle those difficult problems...

Approaches to tackling NP- hard problems:

1. **"this sometimes solve it in OK time"**

2. **"this always answers in OK time, and is close enough"**

How to tackle those difficult problems...

Approaches to tackling NP- hard problems:

1. **Use a strategy that solves the problem exactly but may not find a solution in polynomial time**

2. **Use an approximation algorithm that finds an approximate (sub-optimal) solution in polynomial time**

Exact solutions

Exact solution strategies:

- Exhaustive search (brute force)

  useful only for small instances

- "Intelligent" exhaustive search
  - backtracking

    eliminates some cases from consideration
  - branch-and-bound

    worst case is still exponential *!*

Idea

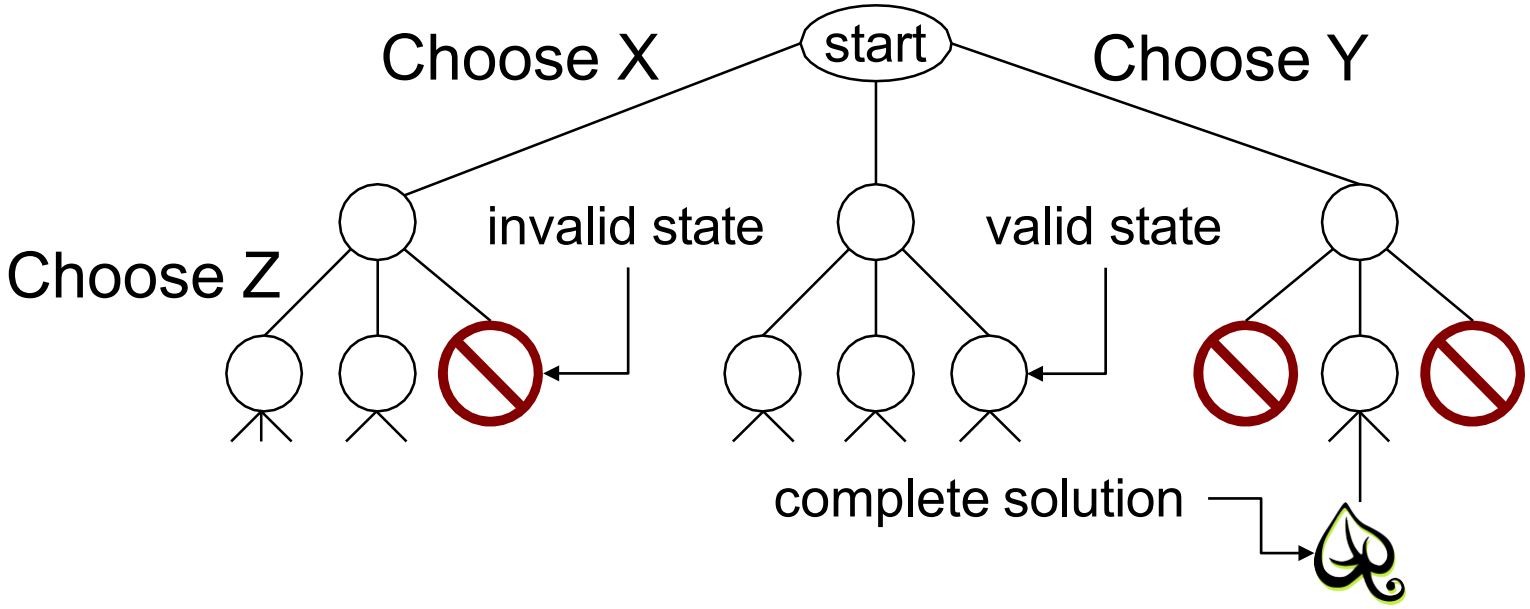
Build candidate solutions one at a time, one step at a time.

Check each ***partially*** constructed solution.

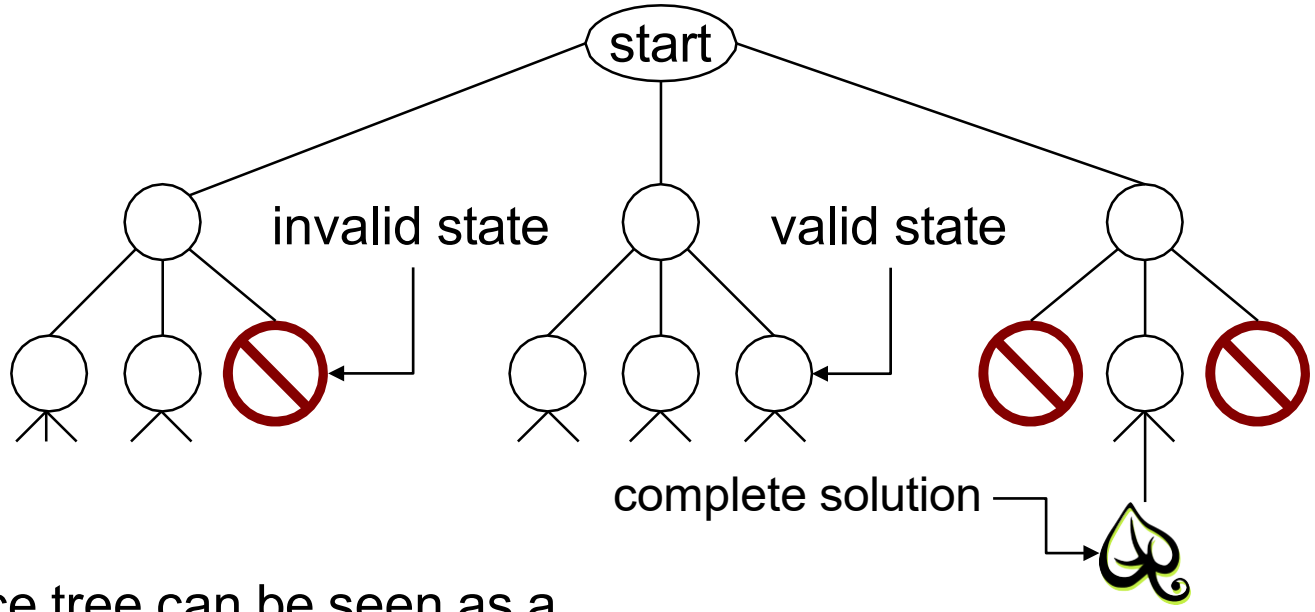
If it cannot lead to a solution, ***stop*** right there.

In the worst case, this leads to exponential explosion (as with exhaustive search)

State Space Tree



State Space Tree



A node in state space tree can be seen as a component of the solution

The complete solution is found at a leaf

Along the path from root to leaf each node must be a component of the solution.

As soon as there is no viable path down the tree the algorithm terminates the search.

Backtracking

Construct the *state space tree*:

- nodes: partial solutions
- edges: choices in completing solutions

Explore the state space tree using depth-first search

"Prune" non-promising nodes

- stops exploring subtree rooted at nodes leading to no solutions and...
- "backtracks" to its parent node

Example: The n-Queen problem

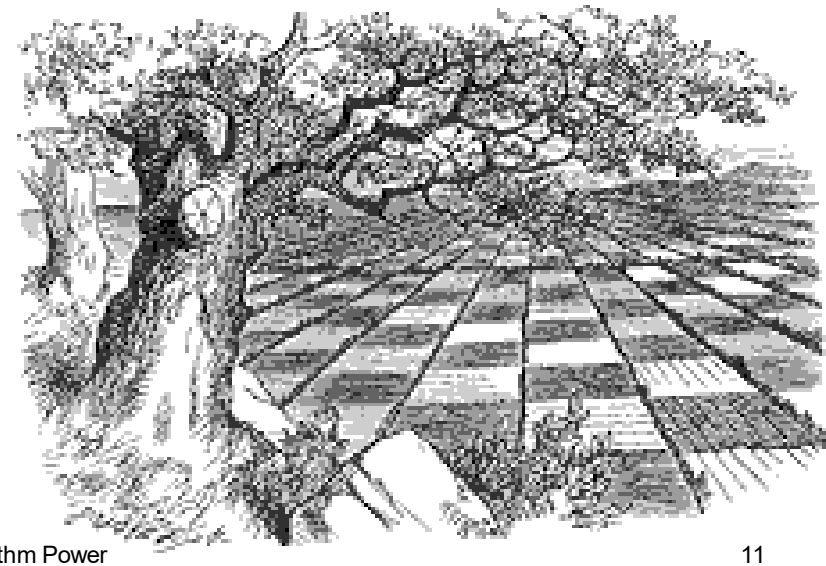
Place n queens on an n by n chess board so that none threaten another

- no two of them are on the same row, column, or diagonal

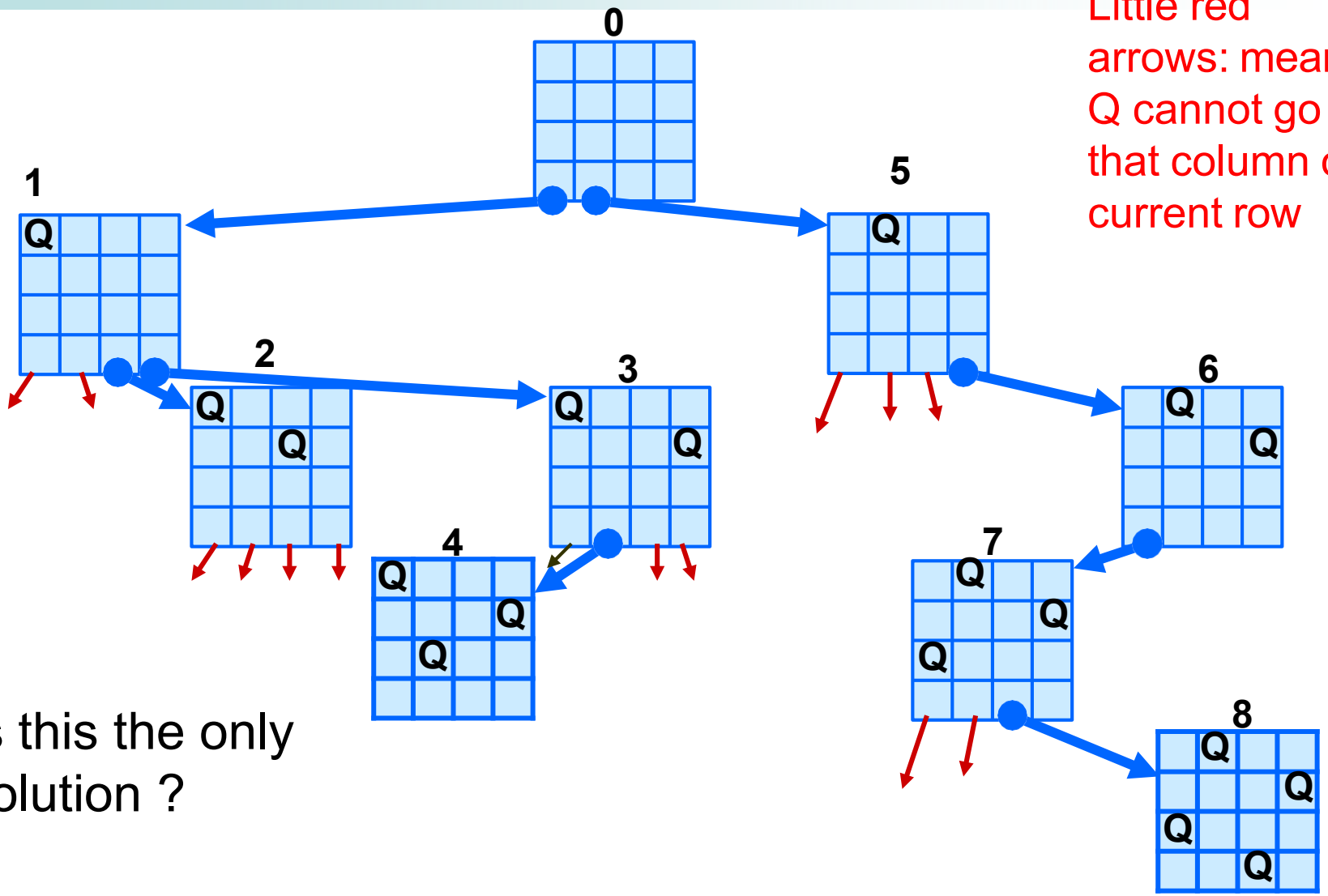
Obviously each must have their own column, so all that remains is to assign a suitable row

- for n=1 solution is trivial
- for n=2 and n=3 there are no solutions

❑ **Exercise: for n=4?**



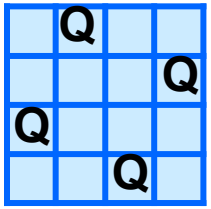
State-Space of the Four-Queens Problem



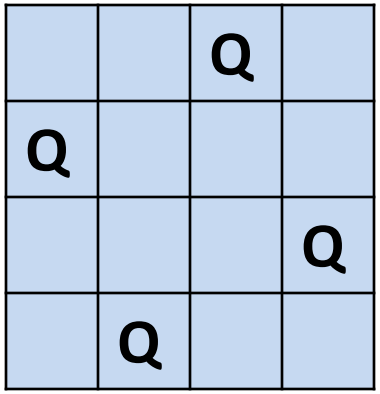
Little red arrows: mean Q cannot go in that column of current row

Is this the only solution ?

Four-Queens Problem

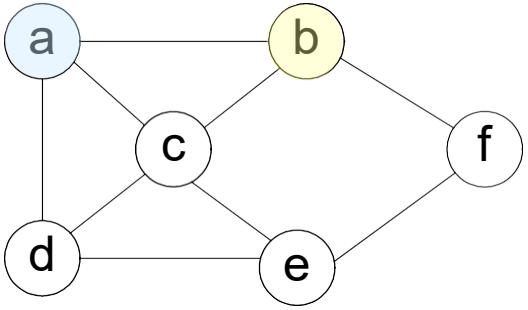


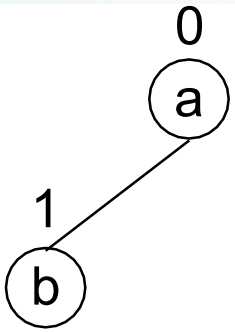
Other solution:



Hamiltonian Circuit Problem I

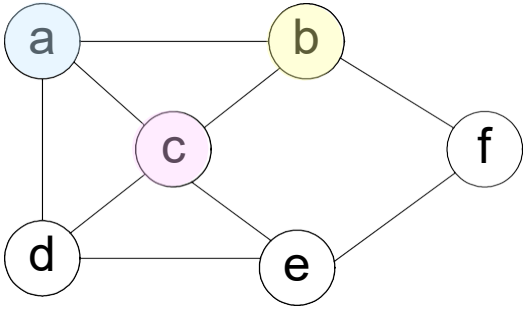
Find a circuit in the given graph such that it visits each node exactly once.

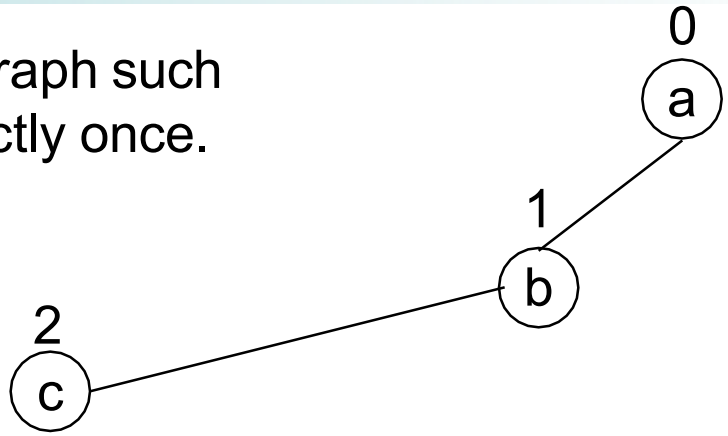




Hamiltonian Circuit Problem II

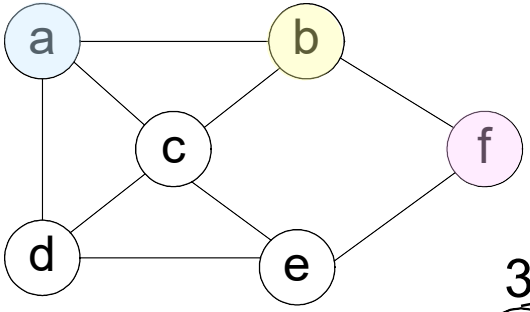
Find a circuit in the given graph such that it visits each node exactly once.

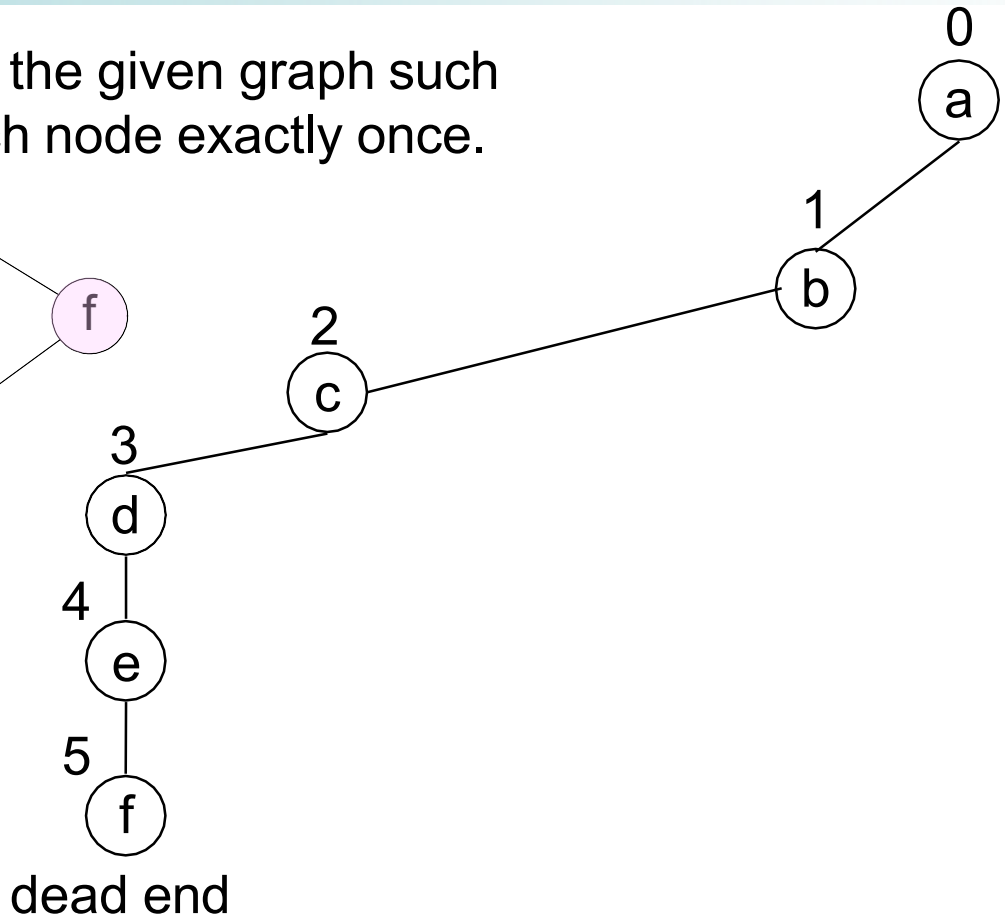




Hamiltonian Circuit Problem III

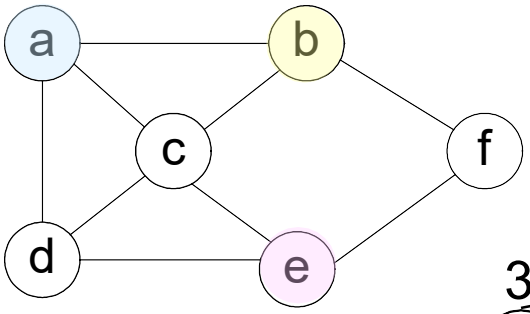
Find a circuit in the given graph such that it visits each node exactly once.

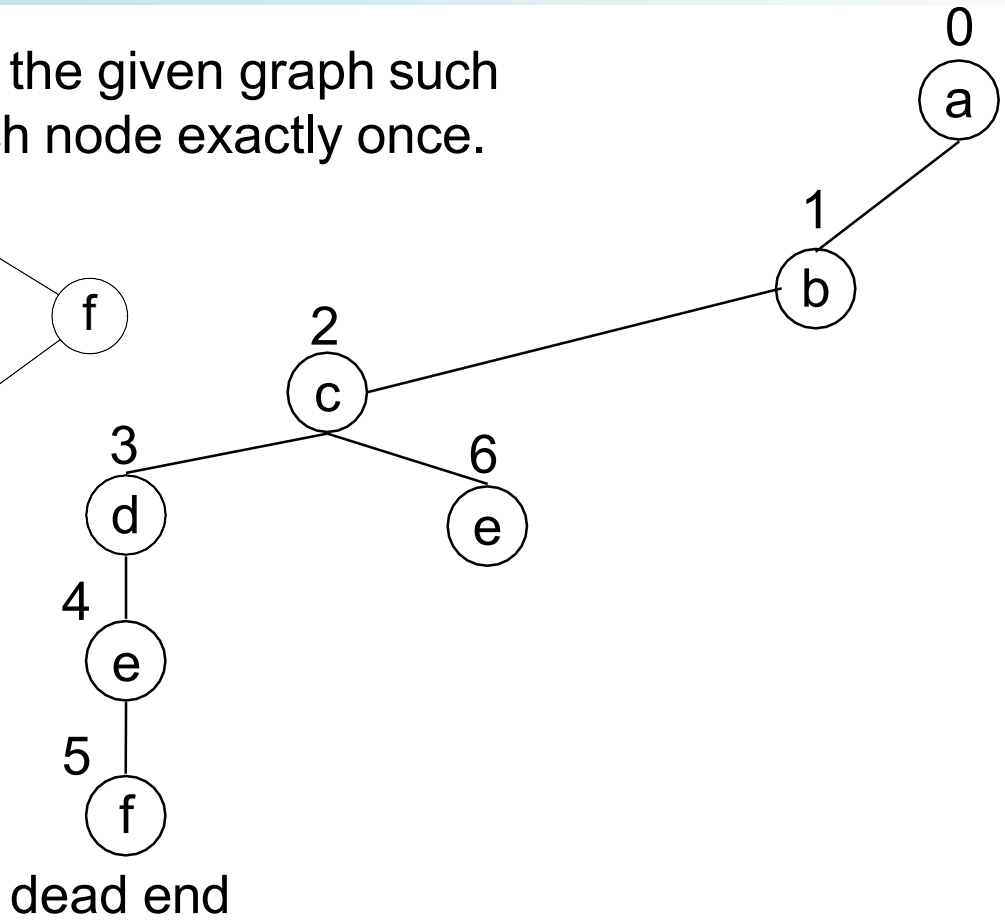




Hamiltonian Circuit Problem IV

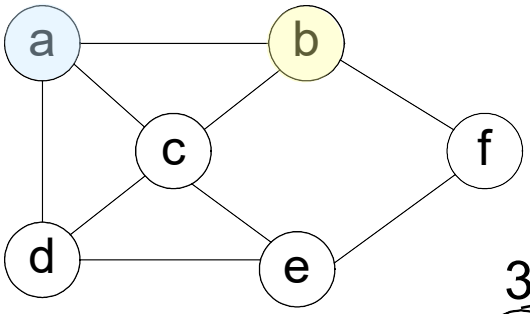
Find a circuit in the given graph such that it visits each node exactly once.

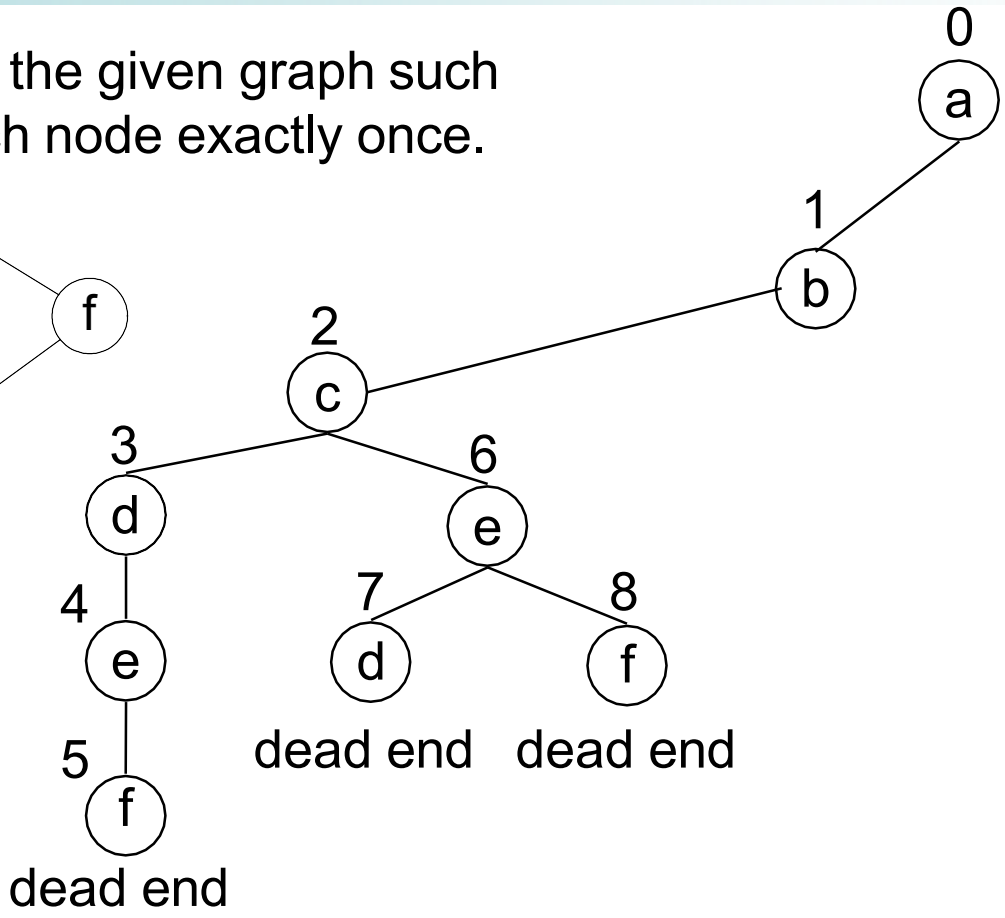




Hamiltonian Circuit Problem V

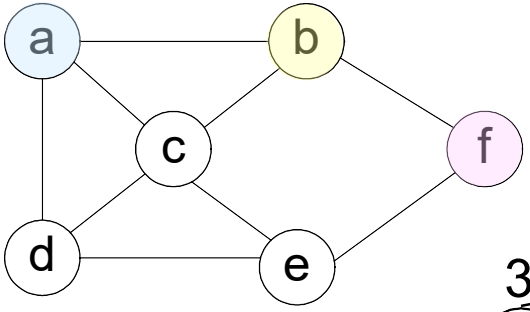
Find a circuit in the given graph such that it visits each node exactly once.

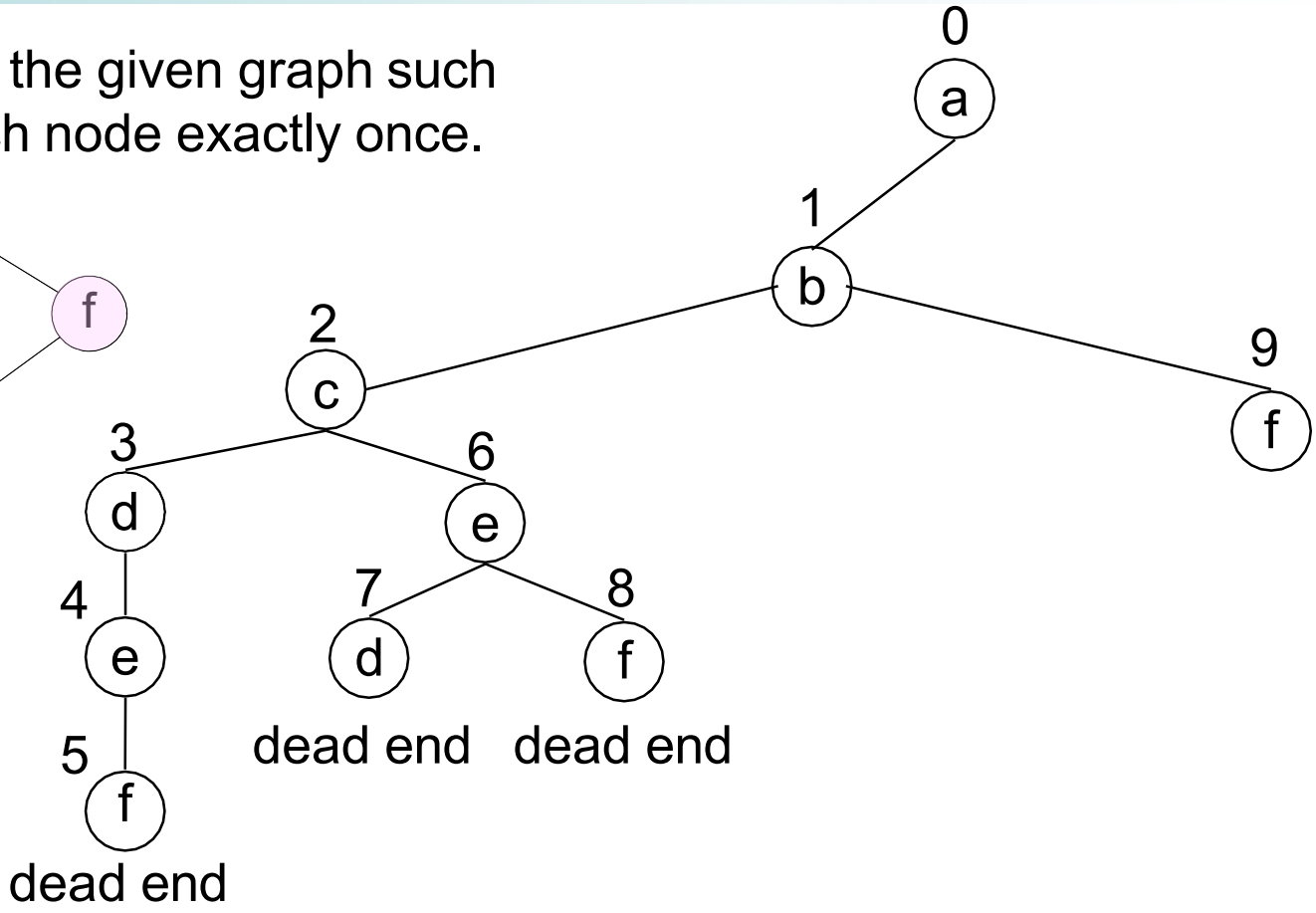




Hamiltonian Circuit Problem VI

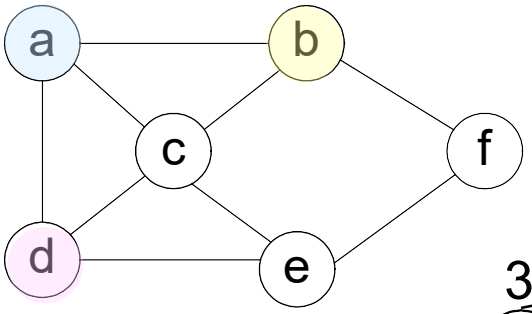
Find a circuit in the given graph such that it visits each node exactly once.

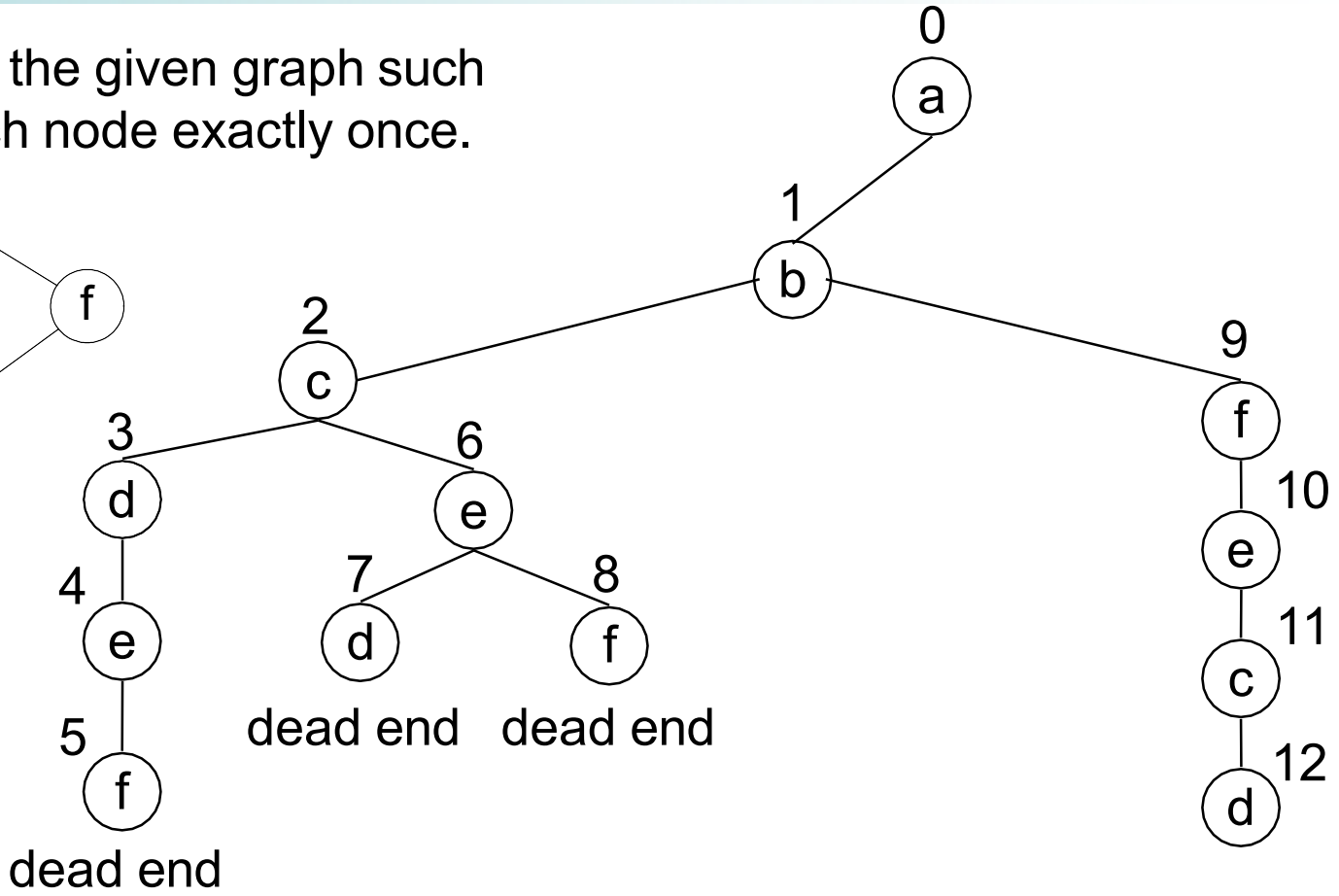




Hamiltonian Circuit Problem VII

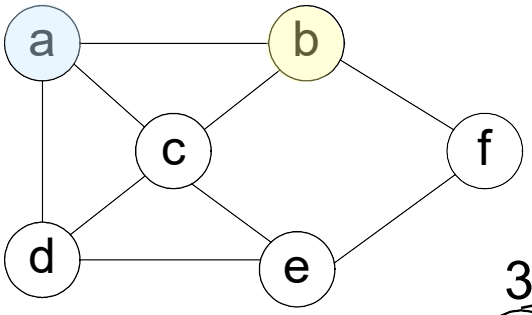
Find a circuit in the given graph such that it visits each node exactly once.

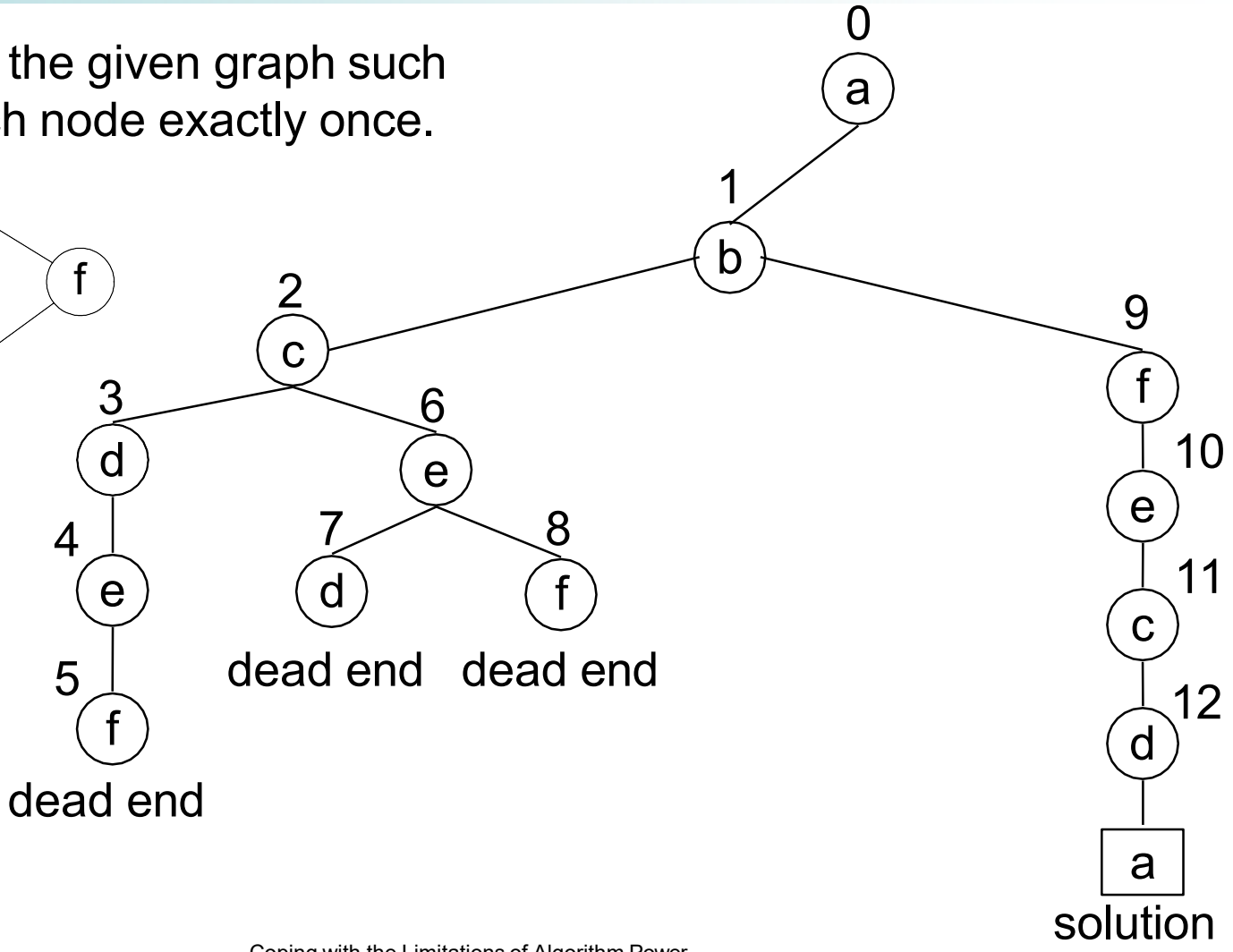




Hamiltonian Circuit Problem VIII

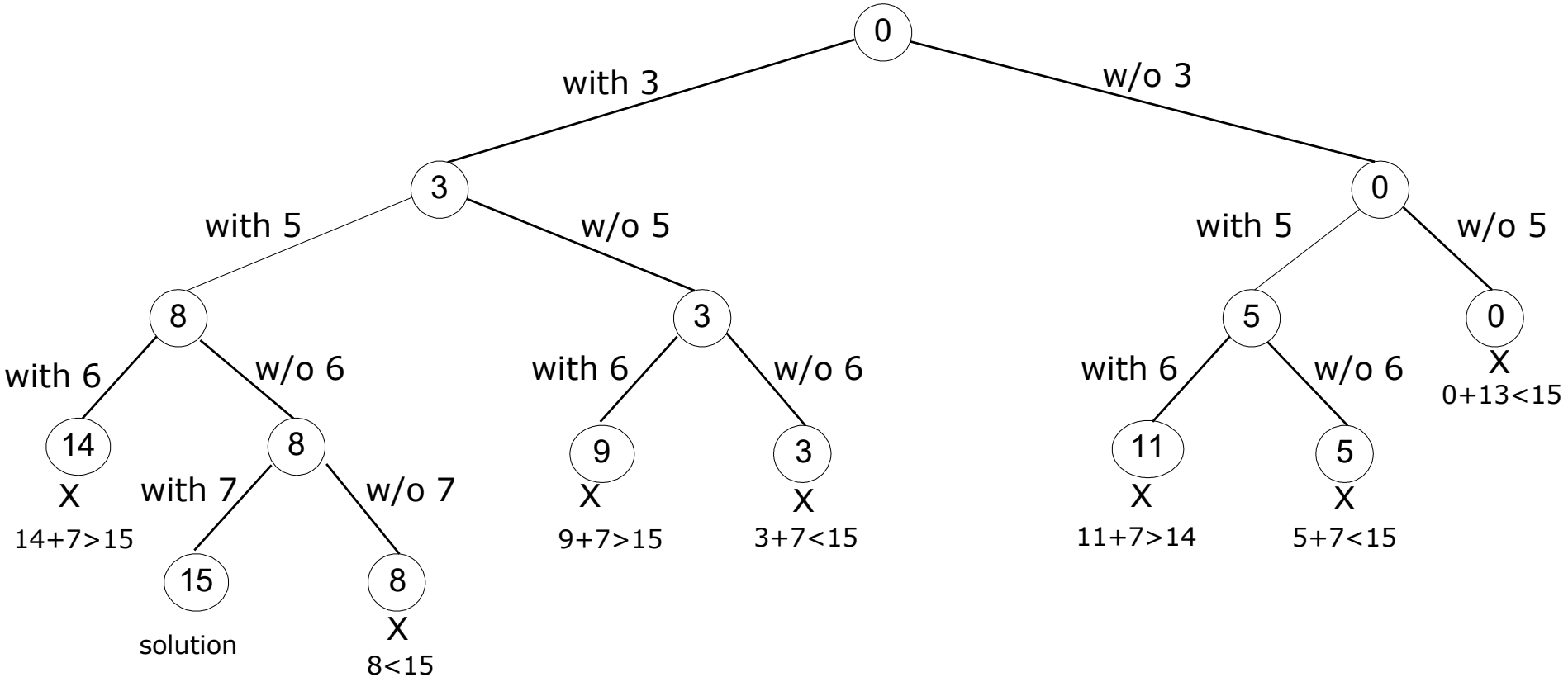
Find a circuit in the given graph such that it visits each node exactly once.





Subset-Sum (w/o means without choosing)

S = {3, 5, 6, 7} and d = 15



Backtracking

If the ultimate solution is a tuple $(x_1, x_2, x_3, \ldots, x_n)$

- Generate a tree whose nodes are valid partial tuples i $(x_1, x_2, x_3, \ldots, x_i)$
- Either the tuple i is a solution or we generate another element $i+1$ $(x_1, x_2, x_3, \ldots, x_i, x_{i+1})$.
- If we cannot find such an x_{i+1} then we backtrack and consider another x_i recursively.

Backtracking

Don't be fooled: we have only considered small examples and this strategy might not work.

- Use only for problems where no efficient algorithm exists (e.g., combinatorial problems)
- At least it is better than exhaustive search … mostly!

  Worst case remains search all possible solutions: efficiency class of the backtracking algorithm remains the same as exhaustive search

Often extra strategies help backtracking even more:

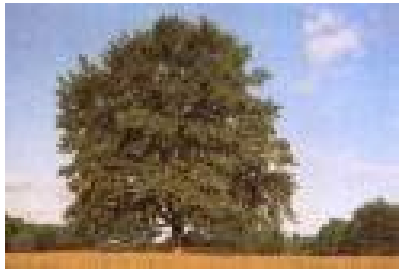
- Exploit symmetry
- Fix certain components of the solution
- Pre-process input e.g. by sorting

Optimization Problems

find a maximum or minimum

constrained (tour length, value of items selected, …)

- feasible solution: satisfies all constraints
- optimal solution: feasible with best value of objective function



Branch-and-Bound — Overview

Backtracking: prune branch if we deduce it does not lead to solution

Compared to backtracking, we need two more things

- bound for every node on the best value that we can get by adding components to partial solution so far
- value of best solution so far

If bound value is not better than best solution seen so far

- the node is non-promising
- no solution obtained can be better than one already known!

Branch-and-Bound — Prune Branch

Three reasons to stop in the current node:

1. Value of node's bound is not better than value of best solution known so far
2. Node represents non feasible solution — constraints are already violated
3. No further choices can be made — we compare value for this solution with that of best solution known so far and update it if needed



Branch-and-Bound — Prune Branch

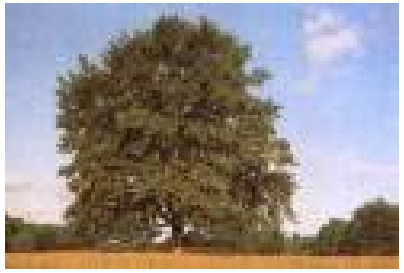
Stop at the current node if:

- no further choices can be made
- proceeding here cannot do better than the best solution so far

So we need:

- the best value solution so far
- a bound for each partial solution





Branch-and-Bound : optimisation

Backtracking:

prune branch if we deduce it does not lead to a solution

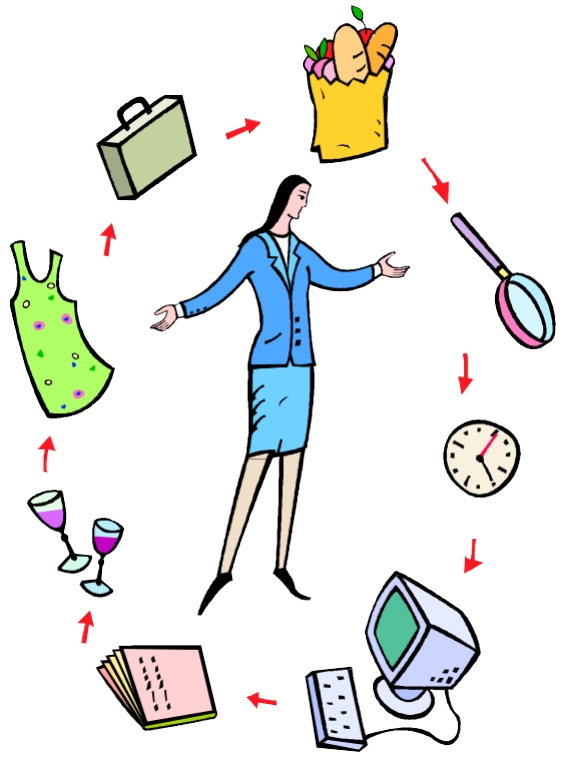
Branch-and-Bound:

prune branch if we deduce it does not lead to a ***better*** solution

Assignment Problem

Assigning *n* people *n* jobs so that total cost of the assignment is as small as possible

Instance of the problem is defined by *n*-by-*n* matrix



Example: The Assignment Problem

Select one element in each row of the cost matrix C so that:

- no two selected elements are in the same column; and
- the sum is minimized

	Job 1	Job 2	Job 3	Job 4
Person a	9	2	7	8
Person b	6	4	3	7
Person c	5	8	1	8
Person d	7	6	9	4

- **Exercise: Lower bound: Any solution to this problem will have total cost of at least …?**

Assignment Problem: Trivial Lower Bound

Cost of any solution can not be smaller then sum of smallest elements in each row of matrix

This is not the cost of valid solution: just lower bound

- 2+3+1+4=10

9	**2**	7	8
6	4	**3**	7
5	8	**1**	8
7	6	9	**4**

Best-First Branch-and-Bound

Order in which nodes are generated is important.

We generate all the children of most promising node among non-terminated leaves in tree

- ones that are still promising — live

Find most promising node by comparing lower bounds of live nodes

- Of course the best solution might turn out to be elsewhere!

(There is actually a polynomial algorithm, $O(n^3)$, James Munkres, 1957, aka Hungarian method, so solution presented is purely educational)

Example: The Assignment Problem

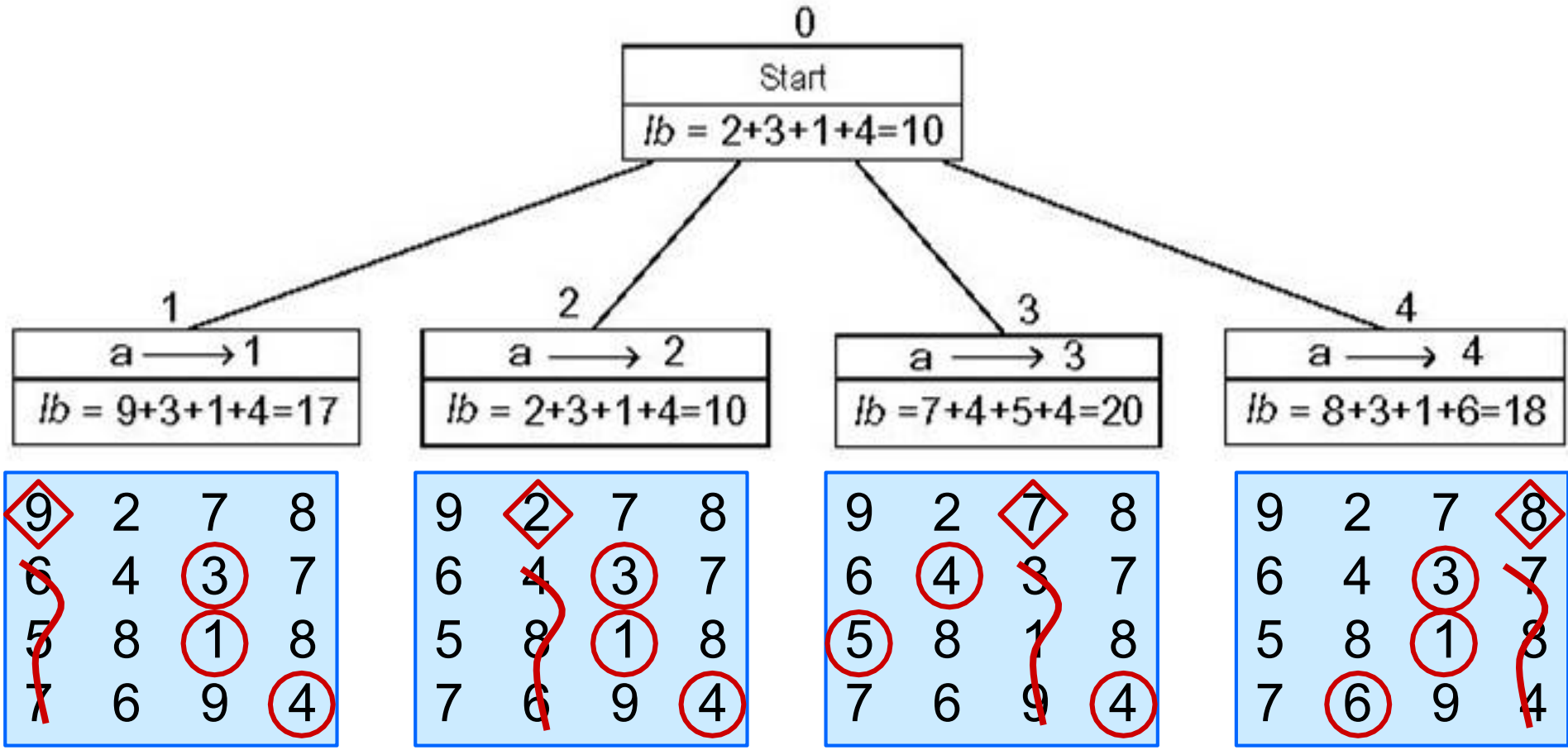
- Systematic state-space-tree search:
  - Each level of the tree => that row of the table
  - A node's children => different elements in that row
- Expand **best** node **next** => one with **smallest** lower bound

	Job 1	Job 2	Job 3	Job 4
Person a	9	2	7	8
Person b	6	4	3	7
Person c	5	8	1	8
Person d	7	6	9	4

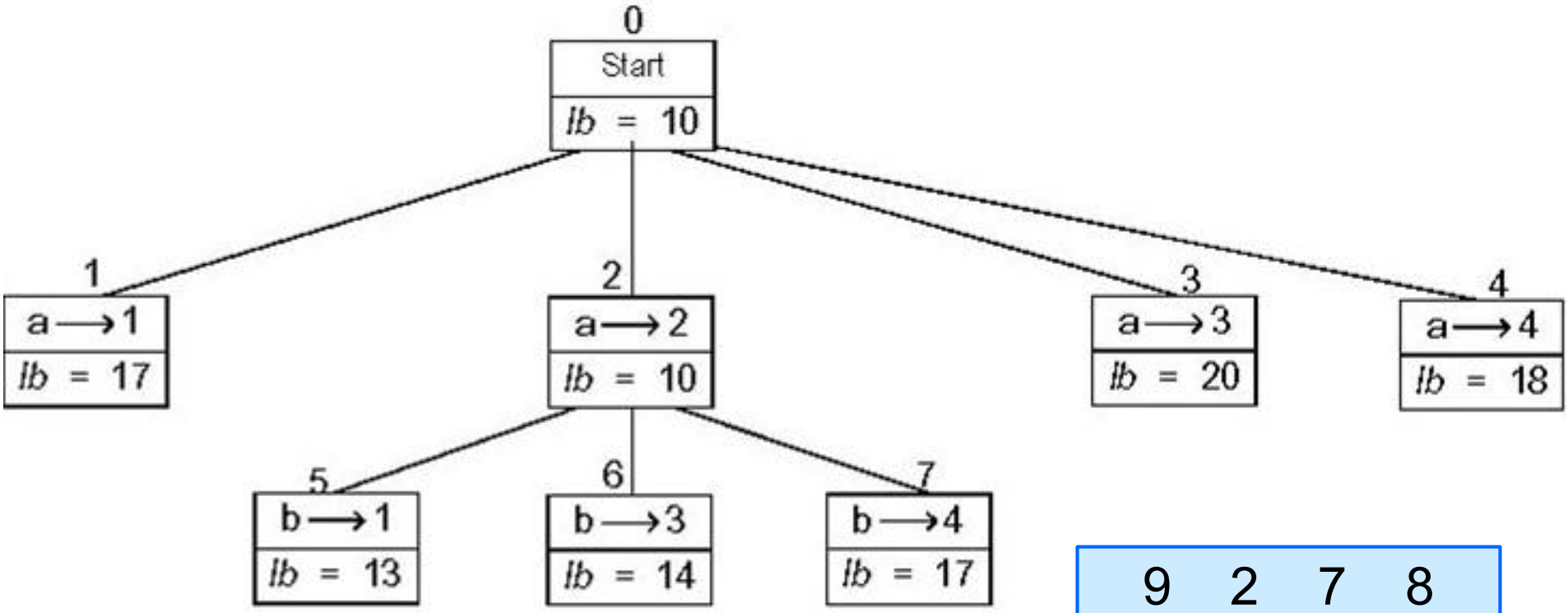
In all examples that follow, **nodes** of the state space trees have a **number above them -** this number shows the **order in which the nodes were generated**.

This is shown because the order of expanding a node is the main strategy of branch and bound – be greedy, do the best (most promising) next

Assignment Problem: Levels 0 and 1 of the state-space tree

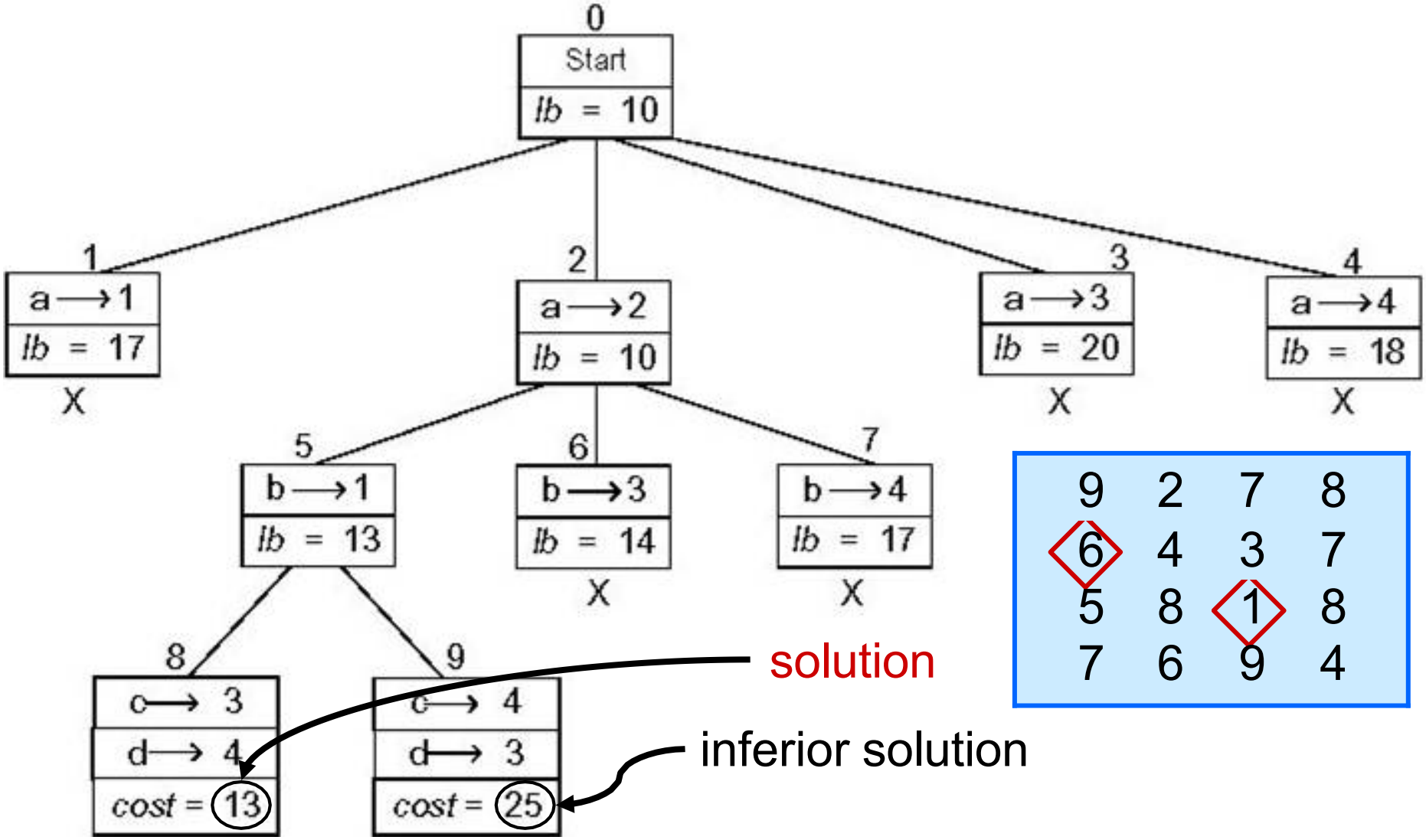


Assignment Problem: Levels 0, 1, 2



9	2	7	8
6	4	3	7
5	8	1	8
7	6	9	4

Assignment Problem: Complete State-Space



Backtracking and Branch-and-Bound

Backtracking

- applies to both optimization and non-optimization problems but preferably the later.
- Uses depth-first search. Searching until reaching the path is no longer viable, then backtrack.

Branch-and-bound

- designed for optimization problems
- Order of nodes generated depends on the strategy used, e.g., best fit.

Travelling Salesman Problem I

Could use branch-and-bound if we come up with lower bound on tour lengths

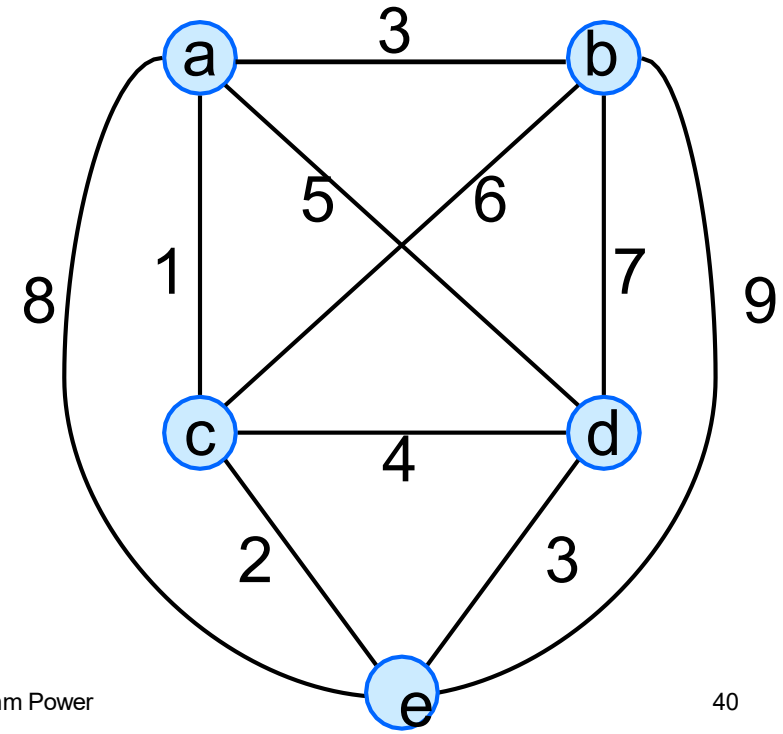
- (trivial) find the smallest distance between two cities and multiply it by number of cities *n*

Better:

- for each city *i*, find sum s_i of distances to two closest cities
- compute sum *s* of those s_i for the *n* cities and divide result *s* by two.

❑ **Exercise: Lower bound …?**

lb **= ⌈[(1+3)+(3+6)+(1+2)+(3+4)+2+3)]/2⌉ = 14**



Travelling Salesman Problem II



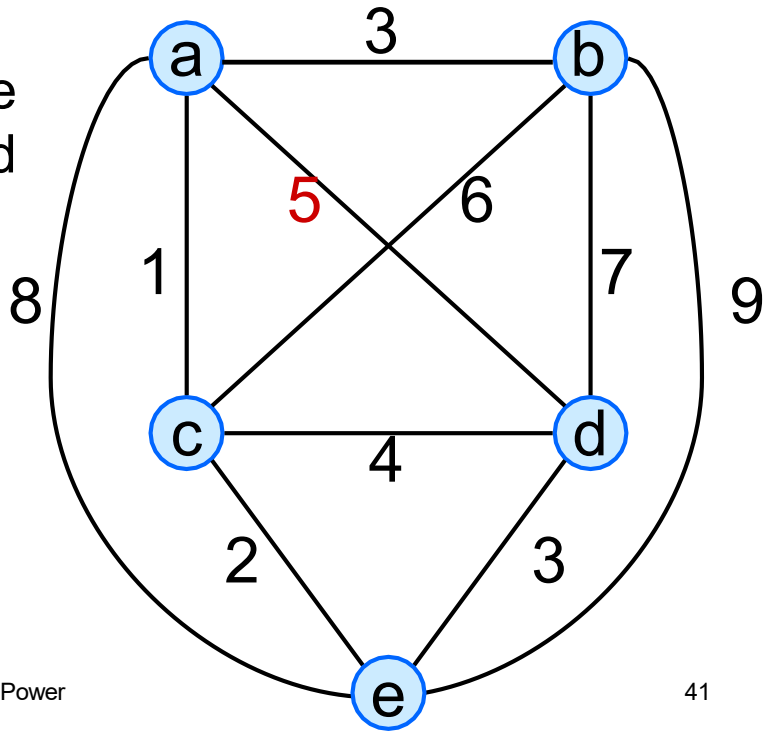
lb = $\lceil [(1+3)+(3+6)+(1+2)+(3+4)+2+3)]/2 \rceil$
= 14

If we know we have to include an edge we can produce a better lower bound

For Example:

- If we must include edge (*a, d*), then we use that when considering nodes *a* and *d.*
- So the lower bound becomes:

= $\lceil [(1+\mathbf{5})+(3+6)+(1+2)+(3+\mathbf{5})+2+3)]/2 \rceil$
= 16



Travelling Salesman Problem III



Now we can start looking for solution

- We assume the tour starts in city *a*
- Graph undirected, so only generate tours where *b* is visited before *c*
- After visiting *n* – 1 cities, tour has no choice but visit remaining city and return to start

Travelling Salesman Problem

Systematic state-space-tree search:

K'th level of the tree => K'th city on the path

A node's children => different roads at that city

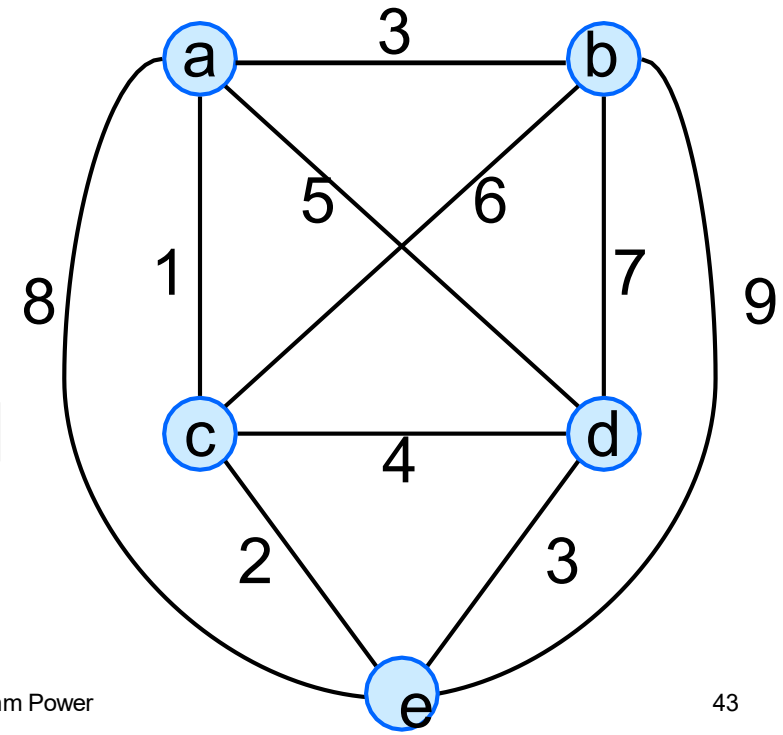
Expand **best** node **next** => one with **smallest** lower bound

lower bound:

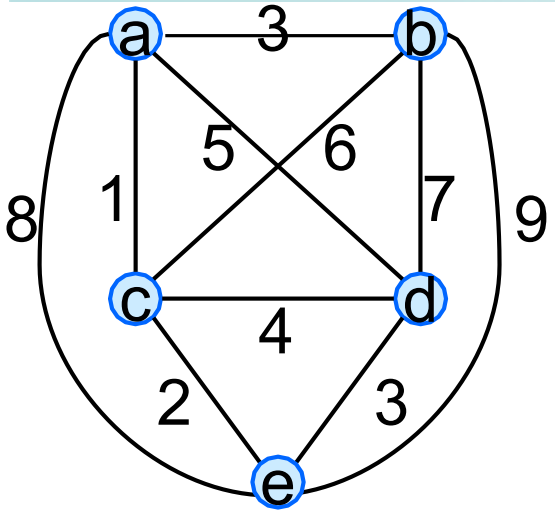
for each city *i*, find sum s_i of distances to two closest cities

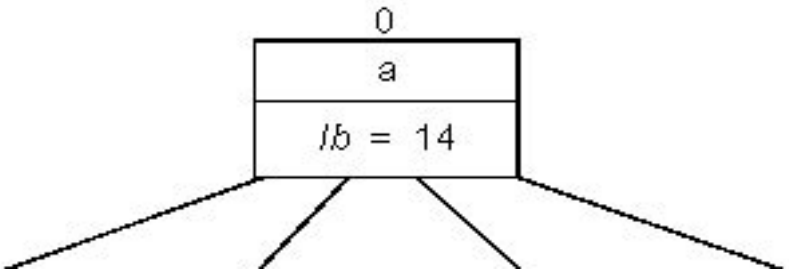
compute sum *s* of those s_i for the *n* cities and divide result *s* by two.

$$lb = \lceil [(1+3)+(3+6)+(1+2)+(3+4)+2+3)]/2 \rceil = 14$$

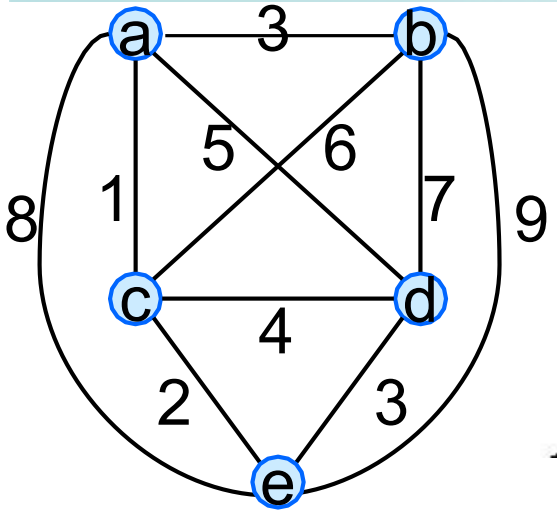


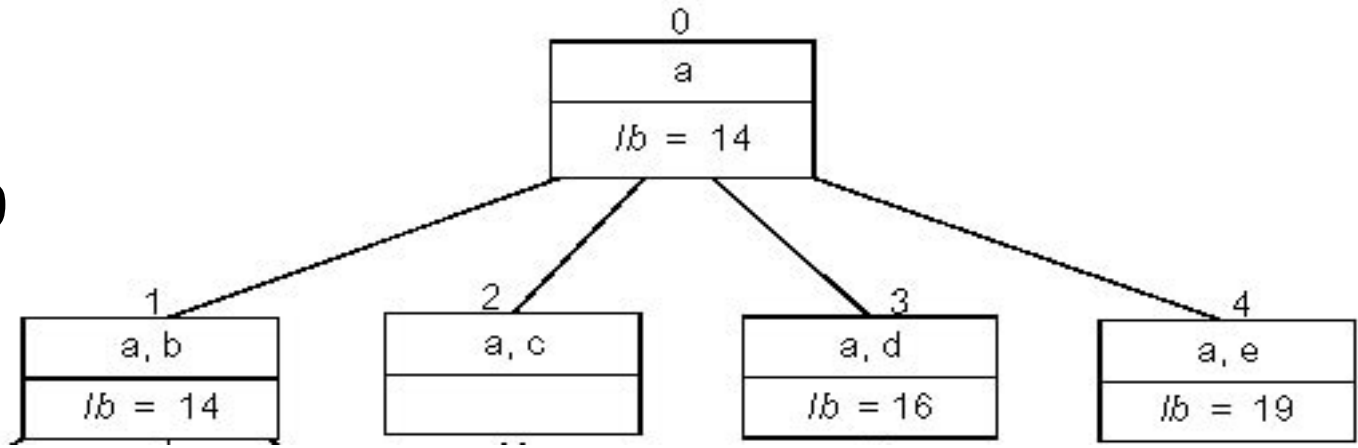
Travelling Salesman Problem IV





Travelling Salesman Problem V

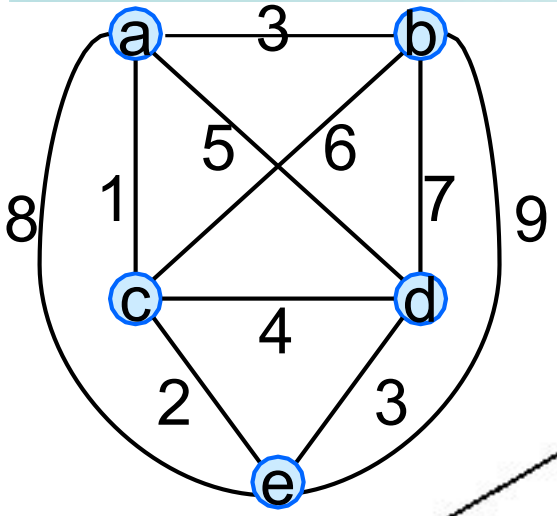


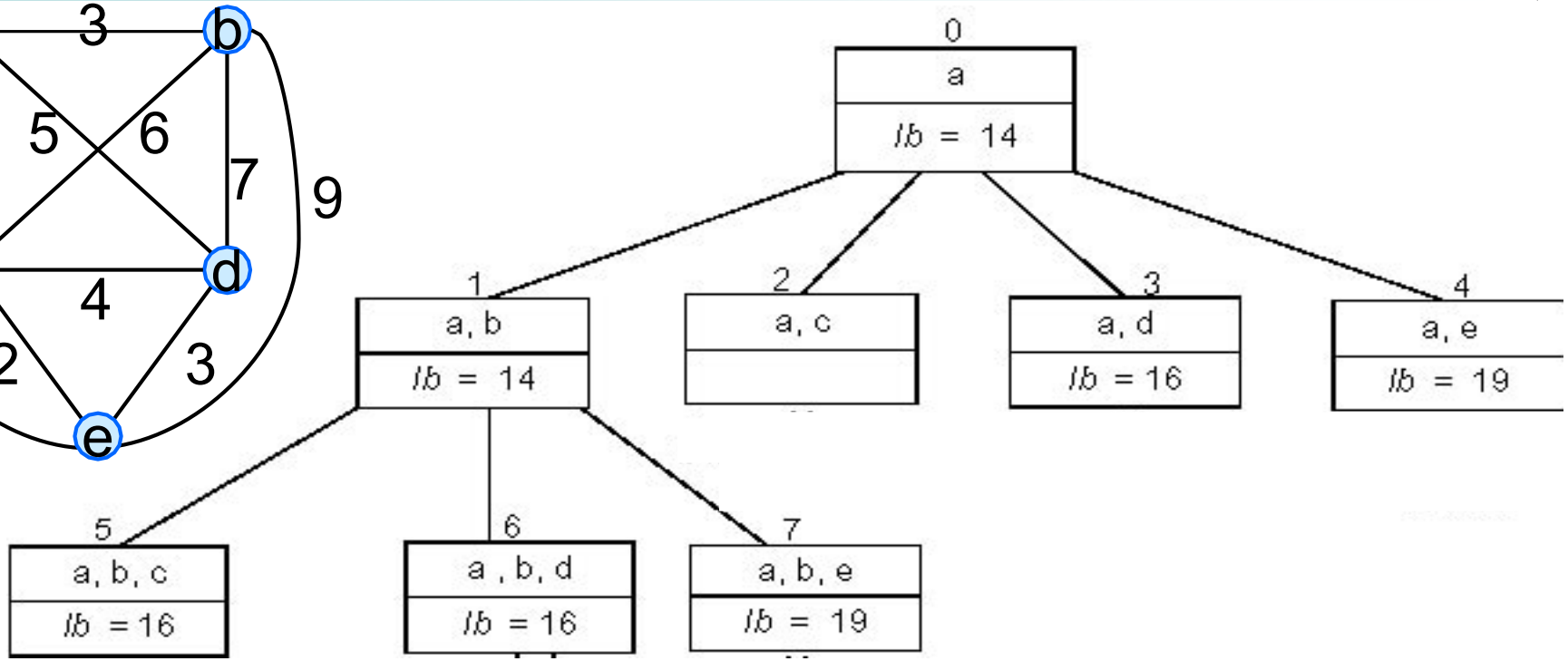


$$lb = \lceil [(1+3)+(3+6)+(1+2)+(3+4)+(2+3)]/2 \rceil = 14$$

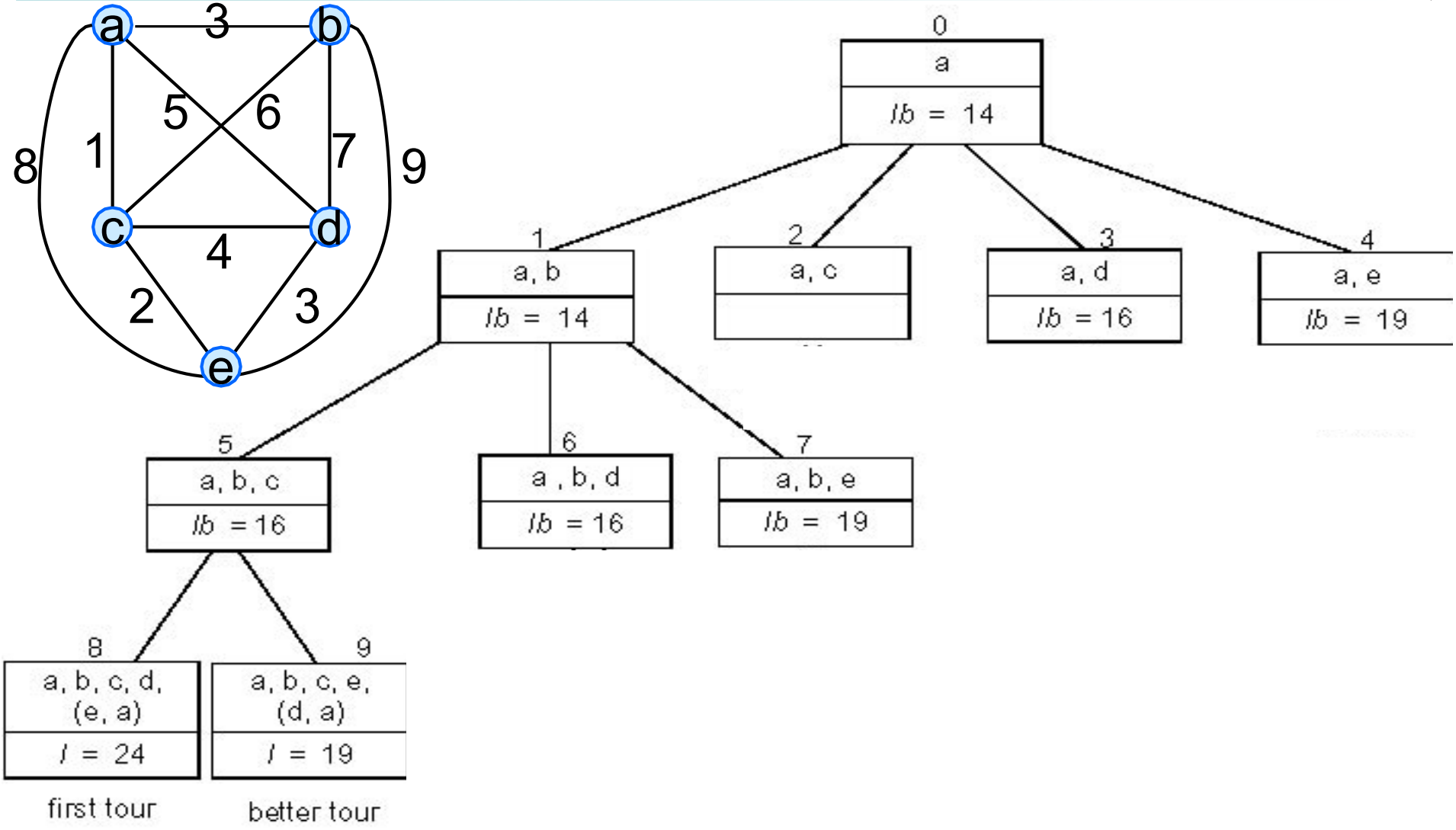
$$lb = \lceil [(1+8)+(3+6)+(1+2)+(3+4)+(2+8)]/2 \rceil = 19$$

Travelling Salesman Problem VI

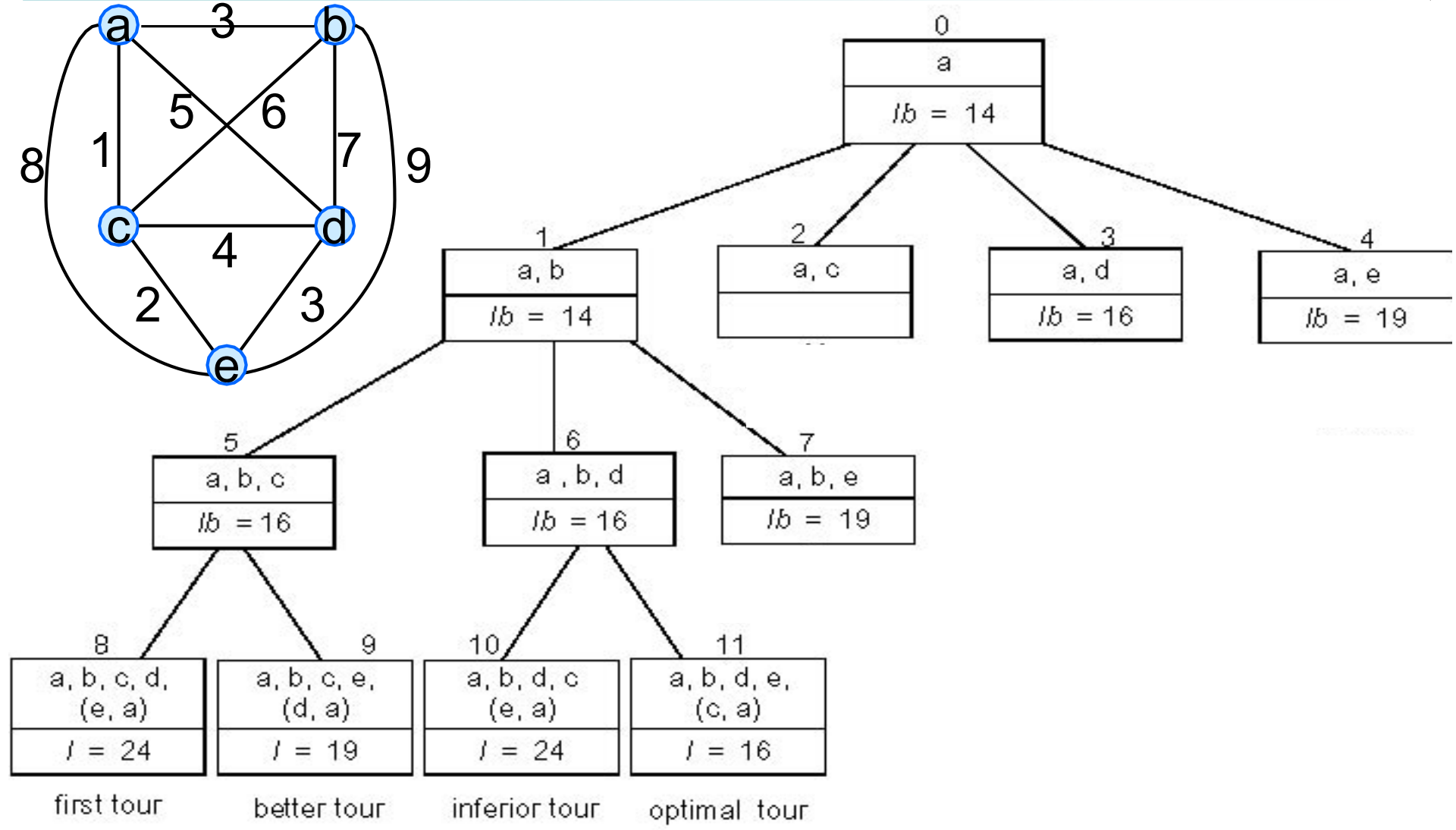




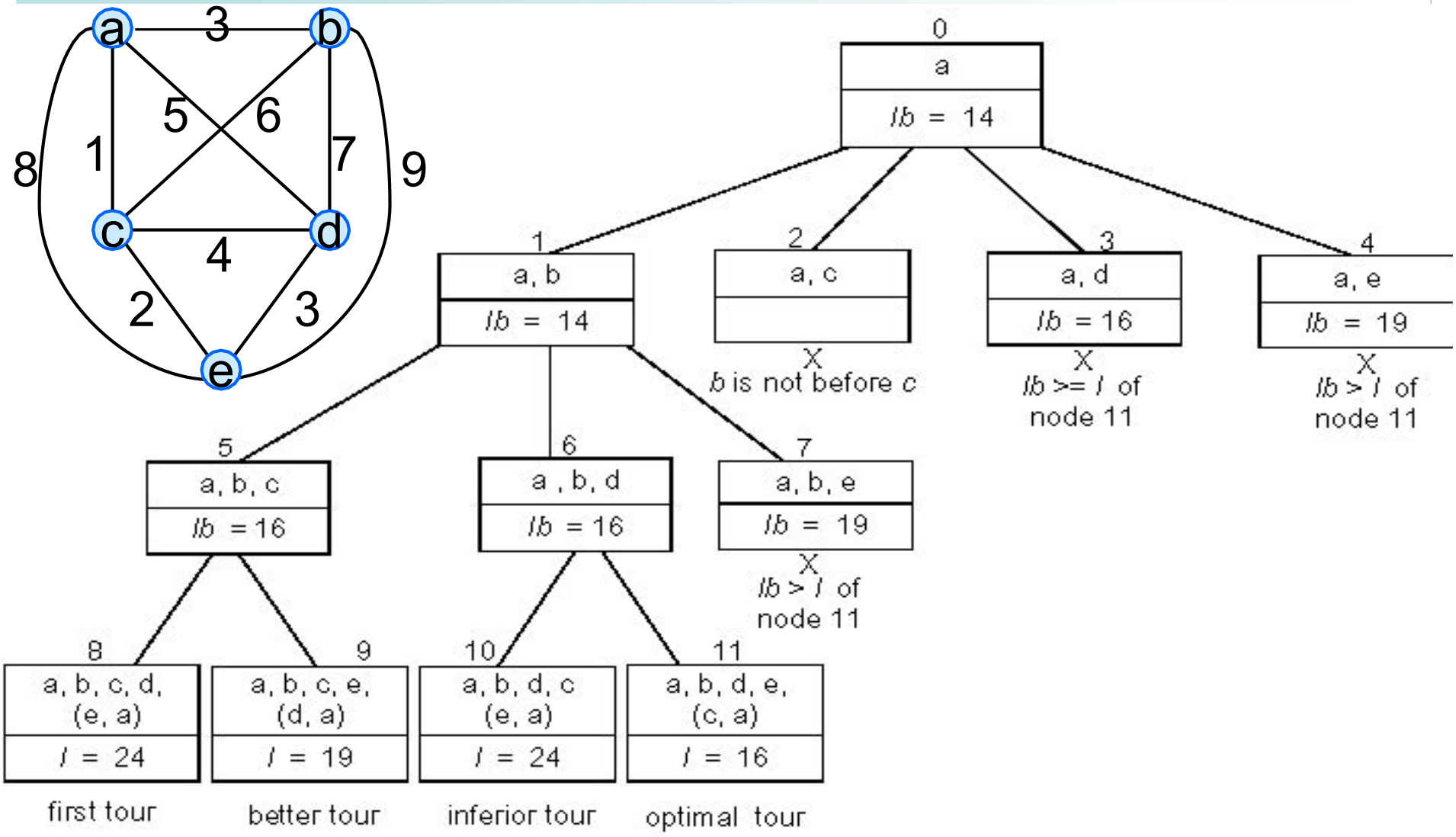
Travelling Salesman Problem VII



Travelling Salesman Problem VIII



Travelling Salesman Problem IX



Knapsack problem - knapsack capacity is 10

ITEM	WEIGHT	VALUE	VALUE/WEIGHT
1	4	$40	10
2	7	$42	6
3	5	$25	5
4	3	$12	4

At the very start, Upper Bound: (capacity) * (best ratio)

As we go along,
Upper Bound: (remaining capacity) * (best ratio of remaining items)

Knapsack: Branch and Bound approach

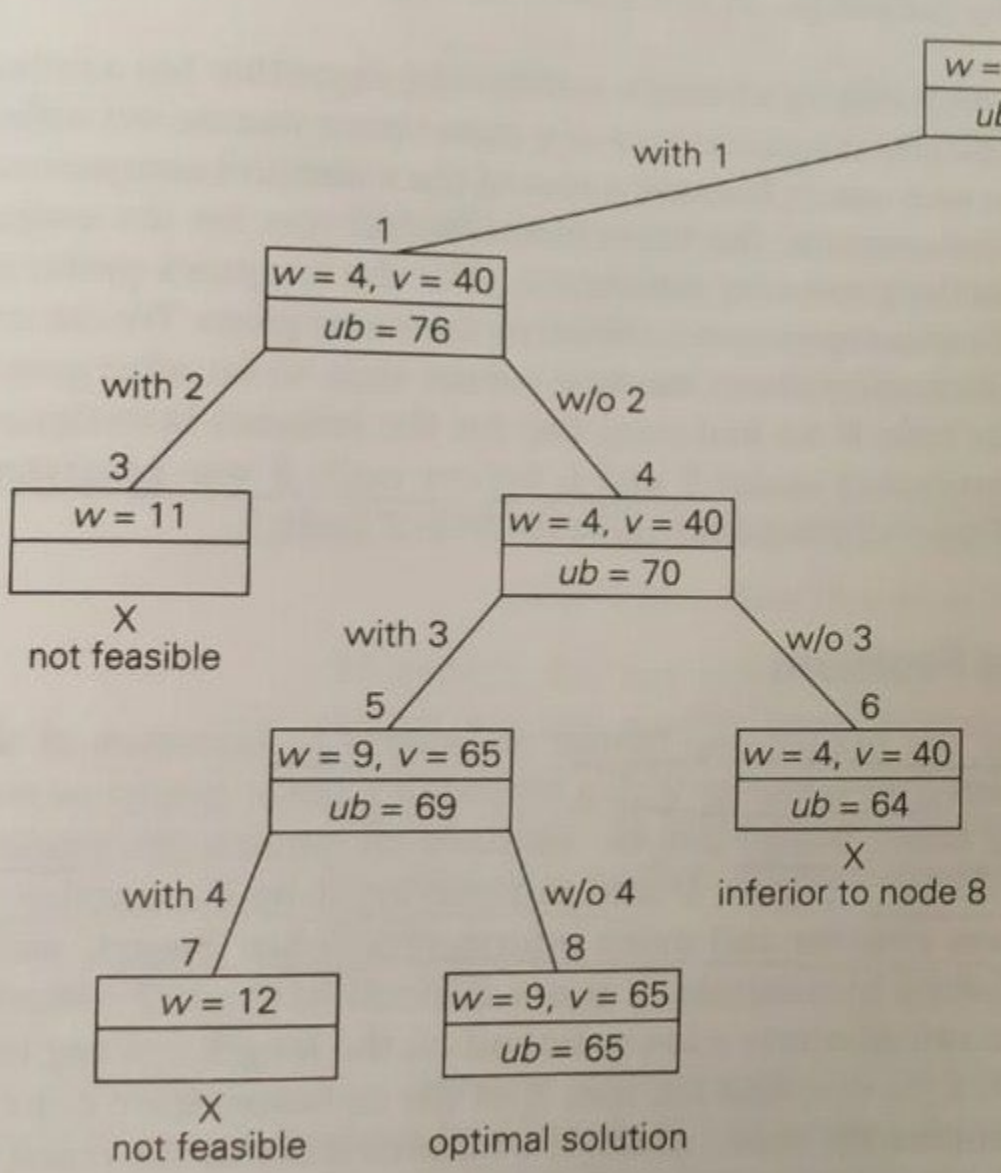
Systematic state-space-tree search:

Each level of the tree => next best value/kg item that fits

A node's 2 children => "take it" and "do not take it"

Expand **best** node **next** => one with **biggest** upper bound

ITEM	WEIGHT	VALUE	VALUE/WEIGHT
1	4	$40	10
2	7	$42	6
3	5	$25	5
4	3	$12	4



ITEM	WEIGHT	VALUE	VALUE/ WEIGHT
1	4	$40	10
2	7	$42	6
3	5	$25	5
4	3	$12	4

FIGURE 12.8 State-space tree of the branch-and-bound algorithm for the instance of the knapsack problem

Limitations

Backtracking's limitations also apply here

Main Point: problem may be solvable in reasonable amount of time and memory, **but** it is not possible to predict.

Can also be improved with knowledge of symmetry to widen the range of search

Branch-and-Bound can be accelerated by a more specific knowledge, such as nontrivial solution already known.

- This can be used as a value for the best solution seen so far.

Branch-and-bound has a choice of order of node generation

To use Branch-and-Bound, it's necessary to formulate a bounding function: non-trivial task in general.

Summary

The two techniques — backtracking and branch-and-bound — both deal with combinatorial problems by constructing "state-space trees."

The difference between them lies in that backtracking is not limited to optimization problems, while branch-and-bound is not restricted to a specific way of traversing the problem's space tree.

Both backtracking and branch-and-bound try to avoid exhaustive search (brute force) by pruning a problem's tree.